

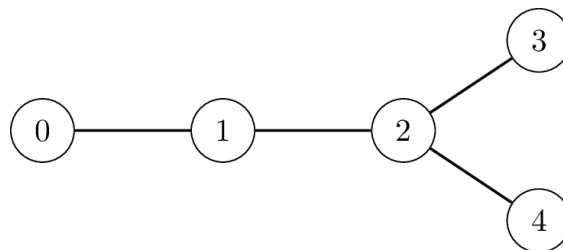
อพยพ

พิพิธภัณฑ์ประวัติศาสตร์ธรรมชาติกำลังศึกษารูปแบบการอพยพของไดโนเสาร์ในโบลิเวีย นักบรรพชีวินวิทยาได้มีการค้นพบรอยเท้าไดโนเสาร์ที่ไซต์สำรวจ N แห่งที่แตกต่างกัน ซึ่งระบุด้วยหมายเลขตั้งแต่ 0 ถึง $N - 1$ โดยเรียงลำดับตามอายุของรอยเท้าไดโนเสาร์จากมากไปน้อย: ไซต์ที่ 0 มีรอยเท้าที่เก่าแก่ที่สุด ในขณะที่ไซต์ที่ $N - 1$ มีรอยเท้าอายุน้อยที่สุด

ไดโนเสาร์ในแต่ละไซต์ (ยกเว้นไซต์ที่ 0) มีการอพยพมาจากไซต์หนึ่งที่มีอายุต่ำกว่า สำหรับแต่ละไซต์ i ที่ $1 \leq i \leq N - 1$ จะมีไซต์ $P[i]$ ที่ต่ำกว่า ($P[i] < i$) เพียงไซต์เดียวที่มีการอพยพของไดโนเสาร์จากไซต์ $P[i]$ ไปยังไซต์ i ไซต์ที่ต่ำกว่าไซต์ใด ๆ อาจเป็นจุดเริ่มต้นของการอพยพไปยังไซต์ที่ใหม่กว่าได้หลายไซต์

นักบรรพชีวินวิทยาได้สร้างแบบจำลองของการอพยพแต่ละครั้งเป็นการเชื่อมต่อแบบไม่มีทิศทางระหว่างไซต์ i และ $P[i]$ สังเกตว่าสำหรับไซต์ x และ y ที่แตกต่างกัน การเดินทางจาก x ไปยัง y นั้นสามารถทำได้ผ่านลำดับของการเชื่อมต่อหลายครั้ง ระยะทางระหว่างไซต์ x และ y ถูกกำหนดให้เป็นจำนวนการเชื่อมต่อขั้นต่ำที่จำเป็นในการเดินทางจาก x ไปยัง y

ตัวอย่างเช่น รูปต่อไปนี้แสดงกรณีที่มีจำนวนไซต์เป็น $N = 5$ โดย $P[1] = 0$, $P[2] = 1$, $P[3] = 2$ และ $P[4] = 2$ เราจะสามารถเดินทางจากไซต์ 3 ไปยังไซต์ 4 ได้โดยผ่านการเชื่อมต่อ 2 ครั้ง ดังนั้นระยะทางระหว่างสองไซต์ดังกล่าวจึงมีค่าเป็น 2



พิพิธภัณฑ์มีความต้องการที่จะหาไซต์ที่มีระยะทางมากที่สุดที่เป็นไปได้

สังเกตว่าไซต์ดังกล่าวไม่จำเป็นต้องมีเพียงคู่เดียว เช่นในตัวอย่างด้านบน ทั้งคู่ $(0, 3)$ และ $(0, 4)$ มีระยะทางเป็น 3 ซึ่งเป็นระยะทางที่มากที่สุด ในกรณีเช่นนี้ ไซต์ใด ๆ ที่มีระยะทางมากที่สุดจะถือเป็นคำตอบที่ถูกต้อง

ในตอนเริ่มต้นเราายัง **ไม่ทราบ** ค่าของ $P[i]$ พิพิธภัณฑ์ส่งทีมวิจัยเข้าสำรวจที่ไซต์ $1, 2, \dots, N - 1$ ตามลำดับ เมื่อไปถึงแต่ละไซต์ i ($1 \leq i \leq N - 1$) ทีมจะดำเนินการสองอย่างต่อไปนี้:

- กำหนดค่าของ $P[i]$ ซึ่งหมายถึงไซต์เริ่มต้นของการอพยพไปยังไซต์ i
- ตัดสินใจว่าจะส่งข้อความ **หนึ่งข้อความ** ไปยังพิพิธภัณฑ์ หรือไม่ส่งข้อความใด ๆ จากไซต์นี้ โดยสามารถพิจารณาจากข้อมูลที่รวบรวมไว้ก่อนหน้านี้

ข้อความจะถูกส่งผ่านระบบสื่อสารดาวเทียมที่มีราคาแพง ดังนั้นแต่ละข้อความจะต้องเป็นจำนวนเต็มระหว่าง 1 ถึง 20 000 รวมหัวท้าย นอกจากนี้ ทีมยังได้รับอนุญาตให้ส่งข้อความรวมทั้งสิ้นไม่เกิน 50 ข้อความ

งานของคุณคือการเขียนโปรแกรมเพื่อสร้างกลยุทธ์ โดย:

- ทีมวิจัยเลือกไซต์ที่จะมีการส่งข้อความพร้อมด้วยค่าจำนวนเต็มของแต่ละข้อความ
- พิพริภันท์สามารถกำหนดคู่ไซต์ที่มีระยะทางสูงสุดได้โดยพิจารณาจากข้อความที่ได้รับจากแต่ละไซต์เท่านั้น โดยพิพริภันท์ทราบว่าแต่ละข้อความถูกส่งมาจากไซต์ใด

การส่งตัวเลขจำนวนมากผ่านดาวเทียมมีค่าใช้จ่ายสูง คะแนนของคุณจะขึ้นอยู่กับทั้งจำนวนเต็มที่สูงที่สุดที่มีการส่งและจำนวนข้อความทั้งหมดที่ทำการส่ง

รายละเอียดการเขียนโปรแกรม

คุณต้องเขียนฟังก์ชันจำนวนสองฟังก์ชัน ฟังก์ชันหนึ่งสำหรับทีมวิจัย และอีกฟังก์ชันหนึ่งสำหรับพิพริภันท์

ฟังก์ชันที่คุณต้องเขียนสำหรับทีมวิจัยคือ:

```
int send_message(int N, int i, int Pi)
```

- N : จำนวนไซต์ที่มีรอยเท้า
- i : หมายเลขของไซต์ที่ทีมวิจัยกำลังสำรวจ
- P_i : ค่าของ $P[i]$
- ฟังก์ชันนี้จะถูกเรียก $N - 1$ ครั้งสำหรับแต่ละกรณีทดสอบ ตามลำดับของ $i = 1, 2, \dots, N - 1$

ฟังก์ชันจะต้องคืนค่า $S[i]$ ซึ่งระบุการดำเนินการของทีมวิจัยที่ไซต์ i :

- $S[i] = 0$: ทีมวิจัยตัดสินใจไม่ส่งข้อความจากไซต์ i
- $1 \leq S[i] \leq 20\,000$: ทีมวิจัยส่งข้อความเป็นจำนวนเต็ม $S[i]$ จากไซต์ i

ฟังก์ชันที่คุณต้องเขียนสำหรับพิพริภันท์คือ:

```
std::pair<int,int> longest_path(std::vector<int> S)
```

- S : อาร์เรย์ความยาว N ซึ่ง:
 - $S[0] = 0$
 - สำหรับแต่ละ $1 \leq i \leq N - 1$ ค่า $S[i]$ คือจำนวนเต็มที่ได้รับมาจาก `send_message(N, i, Pi)`
- ฟังก์ชันนี้จะถูกเรียกเพียงครั้งเดียวสำหรับแต่ละกรณีทดสอบ

ฟังก์ชันจะต้องคืนค่าเป็น `pair` ของไซต์ (U, V) ที่มีระยะทางมากที่สุด

ในการให้คะแนนจริง โปรแกรมที่เรียกใช้ฟังก์ชันด้านบนจะถูกรัน **สองครั้งพอดี**

- ในระหว่างการรันโปรแกรมครั้งแรก:
 - `send_message` จะถูกเรียกเป็นจำนวน $N - 1$ ครั้ง
 - โปรแกรมของคุณสามารถทำการจัดเก็บข้อมูลและนำไปใช้ในการเรียกครั้งต่อไป

- ค่าที่ส่งกลับมา (อาร์เรย์ S) จะถูกเก็บไว้โดยระบบตรวจสอบ
- ในบางกรณี เกรดเดอร์จะมีพฤติกรรมแบบ **ปรับตัวได้** (adaptive) ซึ่งหมายความว่าค่าของ $P[i]$ ในการเรียก `send_message` อาจขึ้นอยู่กับผลการดำเนินการของทีมวิจัยในการเรียกครั้งก่อน ๆ
- ในระหว่างการรันโปรแกรมครั้งที่ 2:
 - `longest_path` จะถูกเรียกเพียงครั้งเดียว โดยข้อมูลเดียวที่มีให้ `longest_path` คืออาร์เรย์ S ที่ได้จากการรันครั้งแรก

เงื่อนไข

- $N = 10\,000$
- $0 \leq P[i] < i$ สำหรับแต่ละ i ที่ $1 \leq i \leq N - 1$

ปัญหาย่อยและการให้คะแนน

ปัญหาย่อย	คะแนน	เงื่อนไขเพิ่มเติม
1	30	ไซต์ 0 กับไซต์อื่นบางแห่งมีระยะทางมากที่สุดในบรรดาไซต์ทั้งหมด
2	70	ไม่มีเงื่อนไขเพิ่มเติม

ให้ Z เป็นจำนวนเต็มสูงสุดที่ปรากฏในอาร์เรย์ S และให้ M เป็นจำนวนข้อความที่ส่งโดยทีมวิจัย

ในกรณีทดสอบใด ๆ หากเงื่อนไขต่อไปนี้เป็นจริงอย่างน้อยหนึ่งข้อ คะแนนของโซลูชันของคุณสำหรับกรณีทดสอบนั้นจะเป็น 0 (รายงานว่า `Output isn't correct` ใน CMS):

- มีสมาชิกใน S อย่างน้อยหนึ่งตัวมีค่าไม่ถูกต้อง
- $Z > 20\,000$ หรือ $M > 50$
- ฟังก์ชัน `longest_path` คืนค่าที่ไม่ถูกต้อง

ในกรณีอื่น คะแนนของคุณในปัญหาย่อย 1 จะได้รับการคำนวณดังนี้:

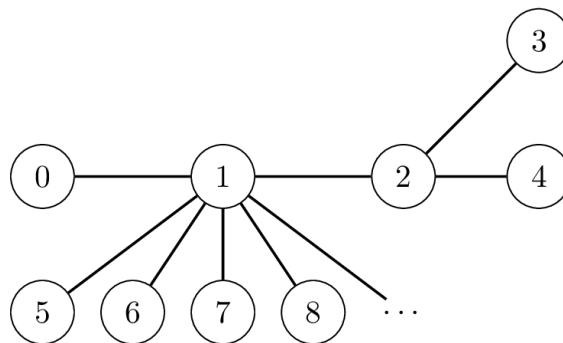
เงื่อนไข	คะแนน
$9\,998 \leq Z \leq 20\,000$	10
$102 \leq Z \leq 9997$	16
$5 \leq Z \leq 101$	23
$Z \leq 4$	30

คะแนนของปัญหาย่อย 2 จะได้รับการคำนวณดังนี้:

เงื่อนไข	คะแนน
$5 \leq Z \leq 20\,000$ และ $M \leq 50$	$35 - 25 \log_{4000} \left(\frac{Z}{5} \right)$
$Z \leq 4$ และ $32 \leq M \leq 50$	40
$Z \leq 4$ และ $9 \leq M \leq 31$	$70 - 30 \log_4 \left(\frac{M}{8} \right)$
$Z \leq 4$ และ $M \leq 8$	70

ตัวอย่าง

ให้ $N = 10\,000$ พิจารณาสถานการณ์ที่ $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, $P[4] = 2$ และ $P[i] = 1$ สำหรับ $i > 4$



สมมติว่ากลยุทธ์ของทีมวิจัยคือการส่งข้อความ $10 \cdot V + U$ เมื่อใดก็ตามที่ มีการเปลี่ยนแปลงคู่ไซต์ (U, V) ที่มีระยะทางสูงสุดหลังทราบผลจากการเรียก `send_message`

ในตอนเริ่มต้น คู่ที่มีระยะห่างสูงสุดคือ $(U, V) = (0, 0)$ พิจารณาลำดับในการเรียกฟังก์ชัน `send_message` ต่อไปนี้ ในระหว่างการรันโปรแกรมครั้งแรก:

การเรียกฟังก์ชัน	(U, V)	ค่าที่ส่งคืน $S[i]$
<code>send_message(10000, 1, 0)</code>	$(0, 1)$	10
<code>send_message(10000, 2, 1)</code>	$(0, 2)$	20
<code>send_message(10000, 3, 2)</code>	$(0, 3)$	30
<code>send_message(10000, 4, 2)</code>	$(0, 3)$	0

สังเกตว่าในการเรียกที่เหลือทั้งหมดค่าของ $P[i]$ คือ 1 ซึ่งหมายความว่าคู่ไซต์ที่มีระยะทางสูงสุดจะไม่มีการเปลี่ยนแปลง และทีมจะไม่ส่งข้อความใด ๆ เพิ่มเติมอีก

จากนั้น ในการรันโปรแกรมครั้งที่สองจะมีการเรียกดังนี้:

```
longest_path([0, 10, 20, 30, 0, ...])
```

พีพริภณท์อ่านข้อความสุดท้ายที่ส่งโดยทีมวิจัยซึ่งก็คือ $S[3] = 30$ และสรุปได้ว่า $(0, 3)$ คือคู่ไซต์ที่มีระยะทางมากที่สุด ดังนั้นการเรียกนี้จะคืนค่าเป็น $(0, 3)$

สังเกตว่าแนวทางนี้ไม่สามารถทำให้พีพริภณท์ระบุคู่ไซต์ที่มีระยะทางมากที่สุดได้ถูกต้องเสมอไป

เกรดเดอร์ตัวอย่าง

เกรดเดอร์ตัวอย่างจะเรียกทั้ง `send_message` และ `longest_path` ในการรันครั้งเดียวกัน ซึ่งแตกต่างจากเกรดเดอร์จริง

รูปแบบข้อมูลนำเข้า:

```
N
P[1] P[2] ... P[N-1]
```

รูปแบบข้อมูลส่งออก:

```
S[1] S[2] ... S[N-1]
U V
```

สังเกตว่าคุณสามารถใช้เกรดเดอร์ตัวอย่างกับค่า N ค่าใดก็ได้